
Table of Contents

=====	1
=====	4
Step 2: Automated Sweep and optimizing parameters	6
=====	8
Defining Angles of the objects	10
=====	13

=====

=====

PART 1: Creating a linear framework for an optical system to propagate rays

=====

```
% Physical meaning of equation 6: represents the matrix that determines the
% distance the ray travels in the x and y direction; formatted as
% [ltar, d; 0, d] for each direction.
% equations 8-10: 8. exit angle theta2 is determined by formula:
% theta2 = -y/f (or -x/f). This can be used to determine how the matrix
% changes during the operation. This will show how the lens changes the
% angle of our rays. This can shown as the matrix [1, 0; -1/f, 1] and can
% be distributed across both x and y directions.
```

```
d1 = 0.2;
% initial distance for rays to be propagated
rays = [
    zeros(1,8),0.01*ones(1,8);
    linspace(-pi/20,pi/20,8),linspace(-pi/20,pi/20,8);
    zeros(1,8),zeros(1,8);
    zeros(1,8),zeros(1,8);
];
% Simulation of rays with differing x and y positions as well as differing
% ray positions and angles
M_d1 = [
    1, d1, 0, 0;
    0, 1, 0, 0;
    0, 0, 1, d1;
    0, 0, 0, 1;
];
% Propagation matrix for rays across the original specified distance

rays_step1 = M_d1 * rays;
% product of the rays traveling the intial distance using the propagation
% matrix for d1

% Plot the rays after the first transformation
ray_z = [zeros(1,size(rays,2)); d1*ones(1,size(rays,2))];
```

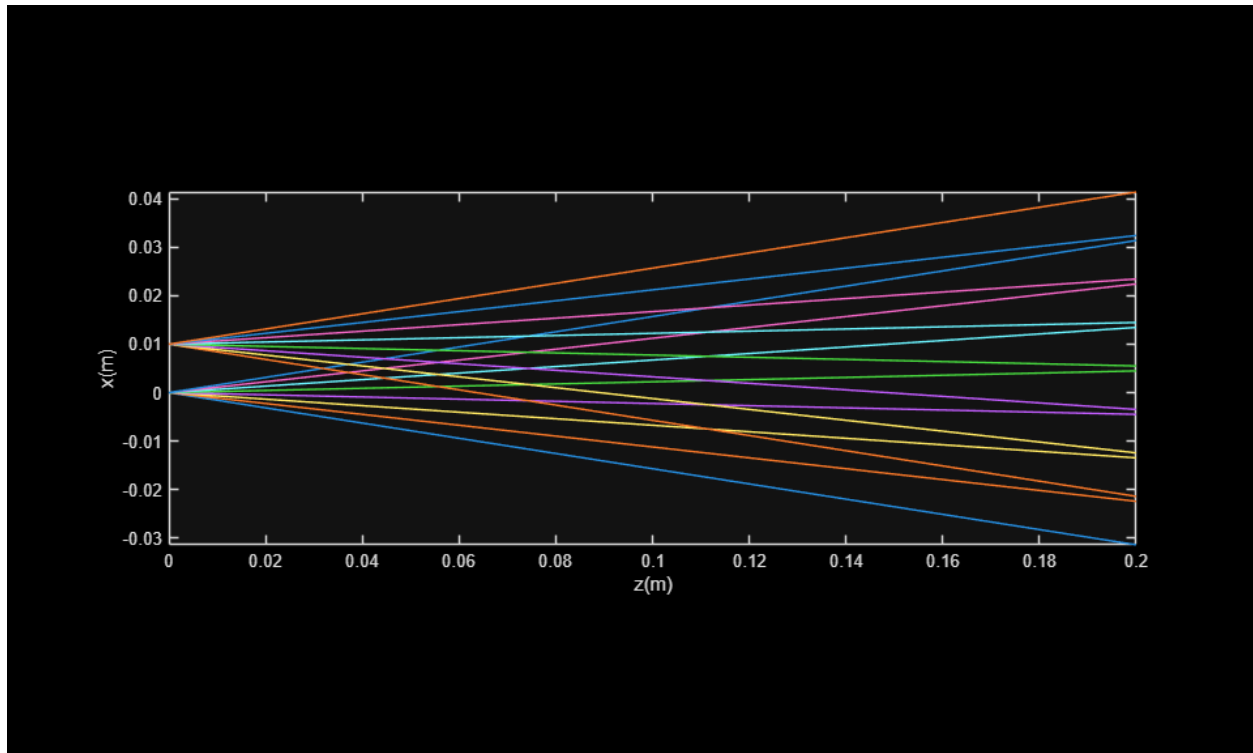
```

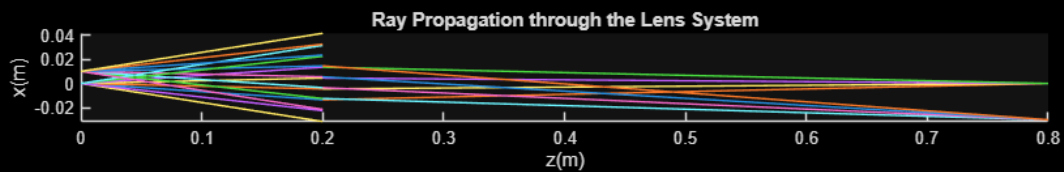
ray_x = [rays(1,:); rays_step1(1,:)];
figure;
plot (ray_z, ray_x);
xlabel('z(m)');
ylabel('x(m)');
axis image;
% Apply the transformation matrix for the exit angle
f = 0.15; % fixed focal length for observation
r = 0.02; % fixed maximum radius for the thin lens
d2 = 0.6;
% Secondary distance the ray traverses after traveling through the lens
M_f = [
    1, 0, 0, 0;
    -1/f, 1, 0, 0;
    0, 0, 1, 0;
    0, 0, -1/f, 1;
];
% Propagation matrix for the thin lens that the rays traverse
idx = abs (rays_step1(1,:)) <= r;
% index for the rays to travel before reaching the maximum radius values to
% eliminate missed plots
rays_hit = rays_step1(:, idx);
% Creates rays that conform to the boundaries set out by the initial index
rays_step2 = M_f * rays_hit;
% Passes the rays that stay within boundaries through the thin lens
% propagation matrix to simulate its affect on the rays
M_d2 = [
    1, d2, 0, 0;
    0, 1, 0, 0;
    0, 0, 1, d2;
    0, 0, 0, 1;
];
% Secondary distance propagation matrix
rays_step3 = M_d2 * rays_step2;
% Propagates the rays that passed through the thin lens with the secondary
% distance

%Plot the steps together:
figure;
hold on;
ray_x_1 = [rays(1,idx); rays_hit(1,:)];
ray_z_1 = [zeros(1,size(rays_hit,2)); d1*ones(1,size(rays_hit,2))];
plot (ray_z_1, ray_x_1);
%This plots the first segment
ray_x_2 = [rays_step2(1,:); rays_step3(1,:)];
ray_z_2 = [d1*ones(1,size(rays_hit,2)); (d1 +
d2)*ones(1,size(rays_step2,2))];
plot(ray_z_2, ray_x_2);
% This plots the second segment
missed_idx= ~idx;
rays_missed = rays_step1(:, ~idx);
ray_x_1 = [rays(1,~idx); rays_missed(1,:)];
ray_z_1 = [zeros(1,size(rays_missed,2)); d1*ones(1,size(rays_missed,2))];
plot (ray_z_1, ray_x_1);

```

```
%This plots the third segment
% Finalize the plot with a legend to distinguish the segments
title('Ray Propagation through the Lens System');
xlabel('z (m) ');
ylabel('x (m) ');
axis image;
hold off;
```





PART 2: Reconstructing a holographic image

```
load('lightField.mat');
sensor_width = 0.005;
num_pixels = 200;
[img,x,y] = rays2img(rays(1,:),rays(3,:), sensor_width, num_pixels);
figure;
imshow(img);
title('Captured Image from Ray Simulation');
axis image;
% It seems as though increaing the amount of pixels within the image causes
% the image to become darker and decreasing it will make the blur look
% blockier. Changing the sensor width will only zoom in or out on the blur.
% The issue isn't the sensor itself, but rather the rays that are
% unfocused.
load ('lightField.mat');

%Define a test propagation distance (d1) in meters
d1 = 0.6;

% Define the Ray Transfer Matrix (M_d1)
M_d1 = [
    1, d1, 0, 0;
    0, 1, 0, 0;
```

```

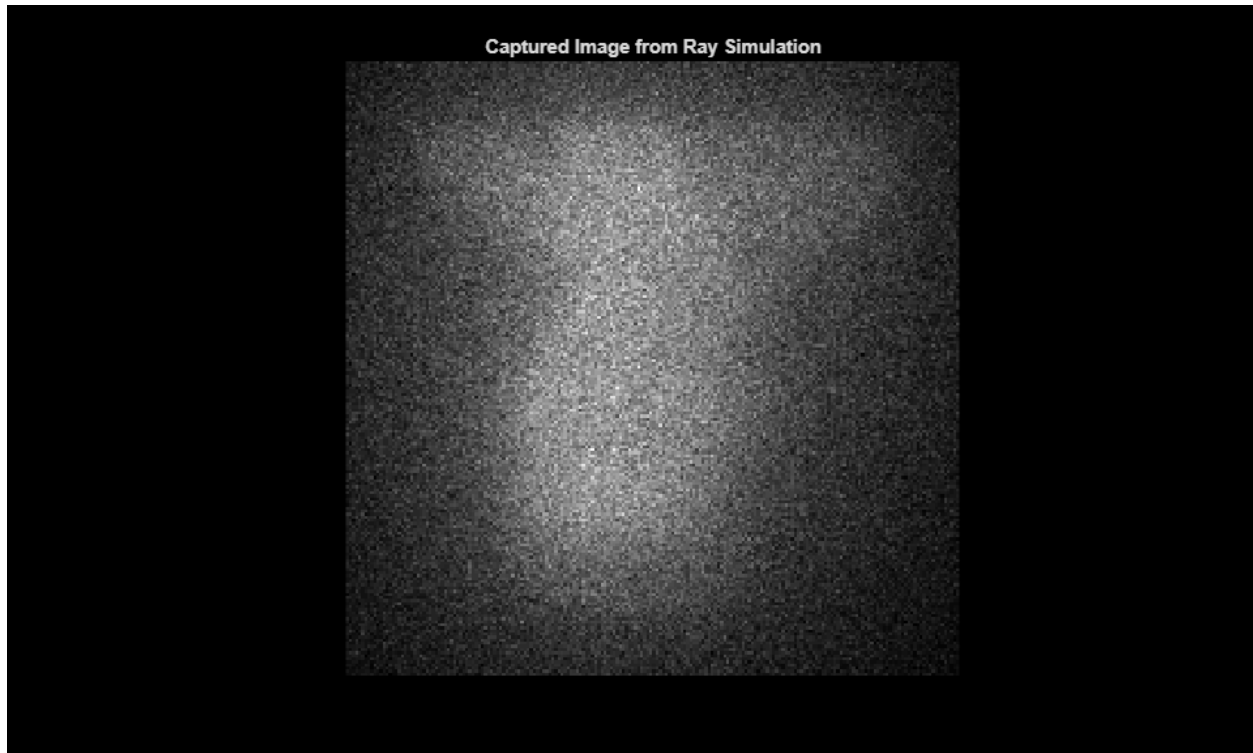
0, 0, 1, d1;
0, 0, 0, 1;
];

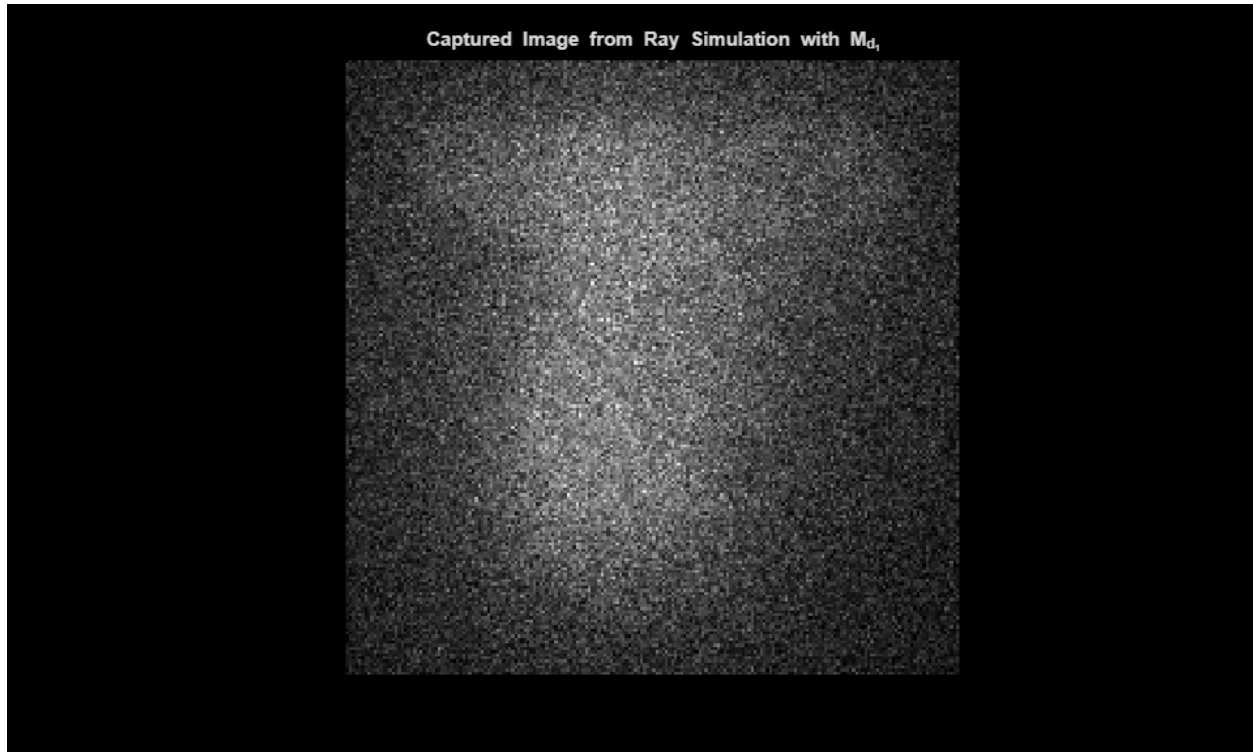
% Apply the linear transformation to all rays
rays_propagated = M_d1 * rays;

%rays_propagated (1,:) = New X positions
%ray_propagated (3,:) = New Y positions
[img,x,y] = rays2img(rays_propagated(1,:),rays_propagated(3,:),
sensor_width, num_pixels);
figure;

%Display the image
imshow(img);
title('Captured Image from Ray Simulation with M_{d_1}');
axis image;
% The M_d1 matrix does not change the blur of the image, it stays
% relativly the same. M_d1 models the rays traveling from d1 to d2, it
% does not "bend" the rays. In the new image, the pixels are just more
% spread out. To form a distinct, sharp image, the full imaging system is
% needed (M_d1, M_f, M_d2).

```





Step 2: Automated Sweep and optimizing parameters

```
% Automated Sweep
load('lightField.mat');
f = 0.08;

% Run the Focus Sweep function (defined at bottom)
[best_d2, peak_score] = RunFocusSweep(f, rays);
% If a valid sensor distance found, compute and report the objects distance
if best_d2 > 0
    d1 = (1/f - 1/best_d2)^(-1);
    fprintf('Found sharp object!\n');
    fprintf('Lens Focal Length f: %.3f m\n', f);
    fprintf('Best Sensor Dist d2: %.3f m\n', best_d2);
    fprintf('Calculated Object Dist d1: %.3f m\n', d1);
else
    % No viable focus peak are detected within the swept range
    fprintf('No sharp image found in range.\n');
end

% Isolate image at the best d2 found
d2 = 0.1000997;
fprintf('Isolating image at d2 = %.5f m...\n', d2);

Mf = [1, 0, 0, 0; -1/f, 1, 0, 0; 0, 0, 1, 0; 0, 0, -1/f, 1];
Md2_best = [1, d2, 0, 0; 0, 1, 0, 0; 0, 0, 1, d2; 0, 0, 0, 1];
```

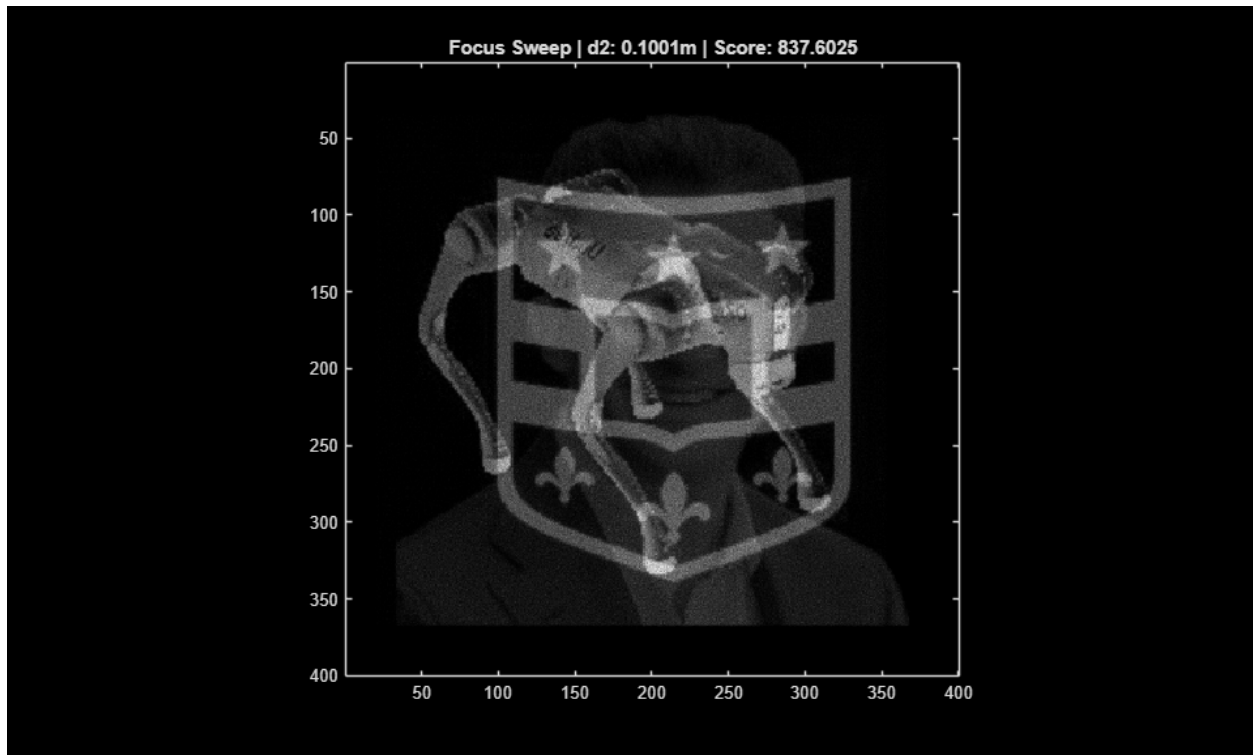
```

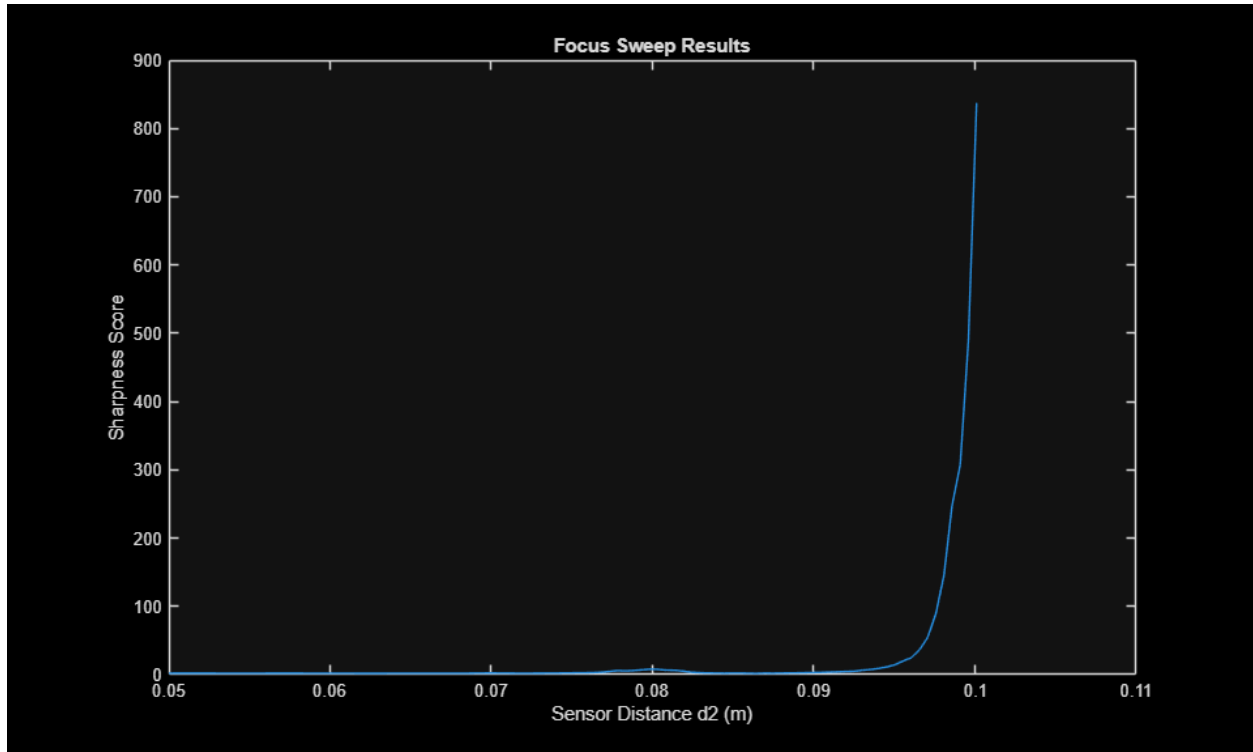
M_final = Md2_best * Mf;
final_rays = M_final * rays;

[img_final, ~, ~] = rays2img(final_rays(1,:), final_rays(3,:), 0.01, 800);

%We are able to somewhat identify the objects emitted by the light rays
(Boston Dynamics Robot Dog, Professor Bruno Sinopoli, Washu Crest),
%however it is still not fully focused as the objects overlap one another.
This is due because the
%lightField.mat containing three objects, so it is not able to entirely
%focus on one single object, resulting in the still blurry images of the
three objects together.

```





```
=====
=====
```

PART 3: ANGULAR FILTERING

Separate the overlapping objects using Theta-X

Step 1: K-means to cluster the rays into three clusters

```
=====
fprintf('Analyzing ray angles to find hidden objects...\n');
angle_data = [rays(2,:) ', rays(4,:) '];

% Run K-Means to find 3 clusters automatically
k = 3;
[cluster_idx, cluster_centers] = kmeans(angle_data, k);
% Utilizes k-means algorithm with 3 clusters to find the optimal positions
% of each angle within the set of rays

all_angles_x = rays(2,:);
% Simulates all angles within the x-position of the dataset

cluster_means = zeros(k, 1);
% Creates a zero vector to hold the angle data of all of the means of the
% clusters
for i = 1:k
    cluster_means(i) = mean(all_angles_x(cluster_idx == i));
    % iterates through the angles from the x-direction with the index from
    % the clustering assignments to find the most optimal centroid values
    % for the data set
```

```

end

% Sorts the angles by ascending to descending order with different colors to
% observe the different values of the angles
[~, sort_order] = sort(cluster_means, 'ascend'); % Ascending order: Left,
Middle, Right
sorted_clusters = sort (cluster_means, "ascend");
colors = {'#bc272d', '#50ad9f', '#e9c716'};
labels = {'WashU Crest', 'Professor Sinopoli', 'Boston Dynamics Robot Dog'};
figure;
hold on;

for i = 1:k
    % Select only the angles belonging to this object
    this_id = sort_order(i);
    cluster_angles = all_angles_x(cluster_idx == this_id);

    % Plot histogram

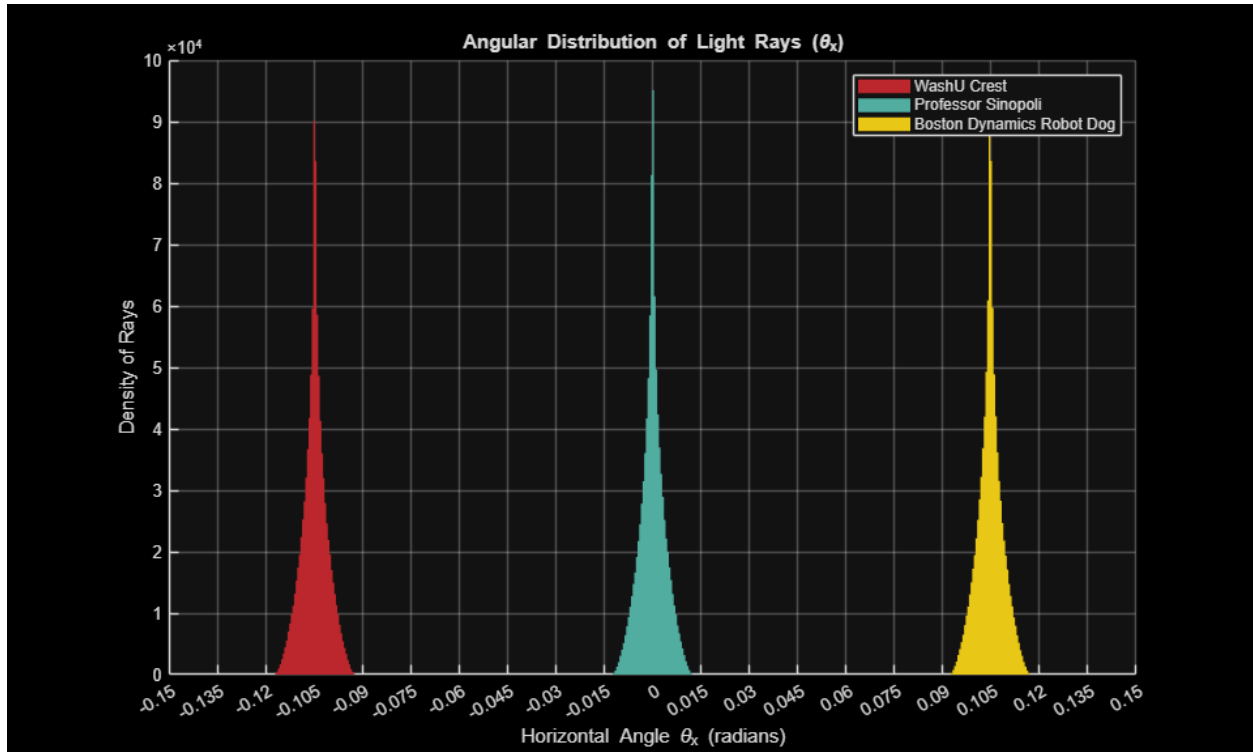
    histogram(cluster_angles, 50, 'FaceColor', colors{i}, ...
        'FaceAlpha', 1, 'EdgeColor', 'none', ...
        'DisplayName', labels{i});
end

% Generates the title, x-label, y-label, and legend for the graph of the
% angles using the k-means algorithm
title('Angular Distribution of Light Rays (\theta_x)');
xlabel('Horizontal Angle \theta_x (radians)');
ylabel('Density of Rays');
legend('show');
grid on;
hold off;

% Sets limits for the graph of the angles found through k-means algorithm
xlim([-0.15, 0.15]);
xticks(-0.15 : 0.015 : 0.15);

Analyzing ray angles to find hidden objects...

```



Defining Angles of the objects

```
% --- 3. RENDER EACH BAND ---

%Define object names corresponding to the sorted clusters (Left, Center,
%Right)
object_names = {'WashU Crest (Left)', 'Professor Sinopoli (Center)',
'Unitree Go2 quadruped robot (Right)'};
sensor_width = 0.15;
num_pixels = 800;

%This loop renders the raw, unprocessed images
figure;
for i = 1:k
    band_id = sort_order(i); %Identifies which cluster is being worked on

    % Creates mask that isolates the rays belonging to each specific
    % cluster
    mask = (cluster_idx == band_id);
    rays_view = final_rays(:, mask);

    %Calculates the max spread of the rays to size the sensor
    x_max = max(abs(rays_view(1,:)));
    y_max = max(abs(rays_view(3,:)));
    final_width = max(x_max, y_max) * 2 * 1.1;

    %Generate raw image
    [img, ~, ~] = rays2img(rays_view(1,:), rays_view(3,:), final_width,
```

```

num_pixels);

    img = imadjust(img,[], [], 0.7);

    subplot(1, 3, i);
    imshow(img);
    title(sprintf('%s\n(Source Angle: %.3f rad)', object_names{i},
sorted_clusters(i)));
end

%This loop renders the final images with the filters
figure;
for i = 1:k
    band_id = sort_order(i);

    mask = (cluster_idx == band_id);
    rays_view = final_rays(:, mask);

    x_max = max(abs(rays_view(1,:)));
    y_max = max(abs(rays_view(3,:)));
    final_width = max(x_max, y_max) * 2 * 1.1;
    %Generates base image
    [img, ~, ~] = rays2img(rays_view(1,:), rays_view(3,:), final_width,
num_pixels);

    %Filters the images
    img = imadjust(img,[], [], 0.7);
    img_denoised = medfilt2 (img, [2 1]);
    img_proc = double(img_denoised) / 255;
    img_smooth = imgaussfilt(img_proc, 3);
    img_final = adapthisteq(img_smooth, 'ClipLimit', 0.01);

    subplot(1, 3, i);
    imshow(img_final);
    title(sprintf('%s\n(Source Angle: %.3f rad)', object_names{i},
sorted_clusters(i)));
end

```

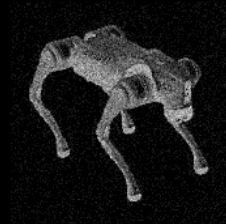
WashU Crest (Left)
(Source Angle: -0.105 rad)



Professor Sinopoli (Center)
(Source Angle: 0.000 rad)



Unitree Go2 quadruped robot (Right)
(Source Angle: 0.105 rad)



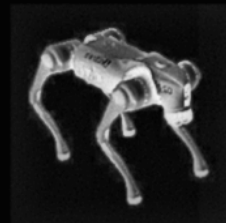
WashU Crest (Left)
(Source Angle: -0.105 rad)



Professor Sinopoli (Center)
(Source Angle: 0.000 rad)



Unitree Go2 quadruped robot (Right)
(Source Angle: 0.105 rad)



=====

=====

HELPER FUNCTIONS

=====

```
function [best_d2, max_score] = RunFocusSweep(f, rays)

    % Search range for sensor distance d2
    d2_range = linspace(0.050, 0.1000997, 100);

    % Sets a search range for the laplacian edge detector scores with a
    % size of the d2 range
    scores = zeros(size(d2_range));

    % Define Lens Matrix
    Mf = [1, 0, 0, 0; -1/f, 1, 0, 0; 0, 0, 1, 0; 0, 0, -1/f, 1];

    for i = 1:length(d2_range)

        % iterates through different values of d2 to find the optimal
        % values
        d2 = d2_range(i);

        % Define Propagation Matrix
        Md2 = [1, d2, 0, 0; 0, 1, 0, 0; 0, 0, 1, d2; 0, 0, 0, 1];

        % Transforms the Rays of the system
        M_system = Md2 * Mf;
        new_rays = M_system * rays;

        % Generates Dynamic Width Calculation by using the changing values
        % of the new_rays with each iteration
        x_max = max(abs(new_rays(1,:)));
        y_max = max(abs(new_rays(3,:)));
        final_width = max(x_max, y_max) * 2 * 1.1;

        % Create Image
        [img, ~, ~] = rays2img(new_rays(1,:), new_rays(3,:), final_width,
400);

        % Calculate Score
        scores(i) = laplacecheck(img);

    end

    % Visualizes the image generated by the focus sweep
    figure;
    imagesc(img); colormap gray; axis image;
    title(sprintf('Focus Sweep | d2: %.4fm | Score: %.4f', d2, scores(i)));
    drawnow;
```

```

    % Finds the peak value of scores and d2
    [max_score, idx] = max(scores);
    best_d2 = d2_range(idx);

    % Plot Results
    figure;
    plot(d2_range, scores);
    xlabel('Sensor Distance d2 (m)');
    ylabel('Sharpness Score');
    title('Focus Sweep Results');
end

function score = laplacecheck(img)
    % Laplacian Edge Detection
    kernel = [0 -1 0; -1 4 -1; 0 -1 0];
    % Sets a laplacian kernel for image processing
    edges = conv2(double(img), kernel, 'valid');
    % Validates the edges by performing convolution on the image and the
    % image processing kernel to find the edges
    score = var(edges(:));
    % Generates the score by finding the variance of the edges in each
    % image and using that value as the scoring metric
end

function [img,x,y] = rays2img(rays_x,rays_y,width,Npixels)
% rays2img - Simulates the operation of a camera sensor, where each pixel
% simply collects (i.e., counts) all of the rays that intersect it. The
% image sensor is assumed to be square with 100% fill factor (no dead
% areas) and 100% quantum efficiency (each ray intersecting the sensor is
% collected).
%
% inputs:
% rays_x: A 1 x N vector representing the x position of each ray in meters.
% rays_y: A 1 x N vector representing the y position of each ray in meters.
% width: A scalar that specifies the total width of the image sensor in
%   meters.
% Npixels: A scalar that specifies the number of pixels along one side of
%   the square image sensor.
%
% outputs:
% img: An Npixels x Npixels matrix representing a grayscale image captured
%   by an image sensor with a total Npixels^2 pixels.
% x: A 1 x 2 vector that specifies the x positions of the left and right
%   edges of the imaging sensor in meters.
% y: A 1 x 2 vector that specifies the y positions of the bottom and top
%   edges of the imaging sensor in meters.
%
% Matthew Lew 11/27/2018
% 11/26/2021 - edited to create grayscale images from a rays_x, rays_y
% vectors
% 11/9/2022 - updated to fix axis flipping created by histcounts2()

```

```

% eliminate rays that are off screen
onScreen = abs(rays_x)<width/2 & abs(rays_y)<width/2;
x_in = rays_x(onScreen);
y_in = rays_y(onScreen);

% separate screen into pixels, calculate coordinates of each pixel's edges
mPerPx = width/Npixels;
Xedges = ((1:Npixels+1)-(1+Npixels+1)/2)*mPerPx;
Yedges = ((1:Npixels+1)-(1+Npixels+1)/2)*mPerPx;

% count rays at each pixel within the image
img = histcounts2(y_in,x_in,Yedges,Xedges);    % histcounts2 for some reason
assigns x to rows, y to columns

% rescale img to uint8 dynamic range
img = uint8(round(img/max(img(:)) * 255));
x = Xedges([1 end]);
y = Yedges([1 end]);

% figure;
% image(x_edges([1 end]),y_edges([1 end]),img); axis image xy;
end

Found sharp object!
Lens Focal Length f: 0.080 m
Best Sensor Dist d2: 0.100 m
Calculated Object Dist d1: 0.398 m
Isolating image at d2 = 0.10010 m...

```

Published with MATLAB® R2025b